

Esercitazione

Esercizio — Thread che si svegliano a cascata

```
int main(void) {  
    pthread_t t1, t2, t3;  
  
    // TODO: creare i 3 thread  
    // pthread_create(&t1, ...)  
  
    // TODO: fare la join dei 3 thread  
  
    // TODO: distruggere mutex e condition  
  
    return 0;  
}
```

Esercitazione

Esercizio — Padre che scrive al figlio e figlio con select() su tastiera

- Scrivere un programma C che:
 - Crea una **pipe**
 - Esegue una fork() per creare un processo figlio
 - Il processo padre invia 10 stringhe al figlio via pipe ad intervalli random
 - Il figlio usa select() per **attendere contemporaneamente** i dati dal padre via socket e dall'utente da tastiera
 - Se arriva dal padre scrive: "[PADRE] <contenuto>«
 - Se arriva dal tastiera scrive: "[UTENTE] <contenuto>"

Esercitazione

Esercizio — Padre che scrive al figlio e figlio con select() su tastiera

```
int main(void) {
    int fd[2];
    if (pipe(fd) == -1) { perror("pipe"); exit(EXIT_FAILURE);}

    pid_t pid = fork();
    if (pid < 0) { perror("fork"); exit(EXIT_FAILURE); }

    if (pid == 0) {
        /* FIGLIO */
        int pipe_fd = fd[0];
        int pipe_closed = 0;
        char buf[MAX_BUF];

        while (1) {
            fd_set readfds;
            FD_ZERO(&readfds);
            // TODO: aggiungere pipe_fd al set
            // TODO: aggiungere stdin (fd 0) al set
```

Test PID namespace

Test - PID = 1 adotta nel namespace

```
int main(void) {

    printf("[P] start:    PID=%d PPID=%d\n", getpid(), getppid());
    pid_t child = fork();
    if (child < 0) {perror("fork"); exit(EXIT_FAILURE);}

    if (child == 0) {
        // FIGLIO
        printf("[C] start:    PID=%d PPID=%d\n", getpid(), getppid());
        for (int i = 0; i < 10; i++) {
            sleep(1);
            printf("[C] t=%d  PID=%d PPID=%d\n", i+1, getpid(), getppid());
        }
        return 0;
    } else {
        // PADRE
        printf("[P] child PID=%d, 3 sec. poi esco...\n", child);
        sleep(3);
        printf("[P] exiting now.\n");
        _exit(0);
    }
}
```

Test PID namespace

Test - PID = 1 adotta nel namespace

Lanciare da terminale

```
./adopt-ppid
```

... adottato da?

reaper è PID = 1, ma esistono
i subreaper

Lanciare dentro il PID namespace

```
sudo unshare --pid --fork --mount-proc bash  
echo $$ # deve stampare 1  
./adopt-ppid
```

Test PID namespace

Test - PID = 1 adotta nel namespace

Test con comandi bash:

```
bash -c 'sleep 100 & exit'
ps -o pid,ppid,cmd | grep sleep
```

shell

```
└─ bash -c 'sleep 100 & exit'      (B)      Quando B esce, S è orfano
   └─ sleep 100                    (S)
```

Lanciare dentro il PID namespace

```
sudo unshare --pid --fork --mount-proc bash
echo $$ # deve stampare 1
bash -c 'sleep 100 & exit'
```

Esercitazione 2

SEZIONE 1 – Pipeline di comandi (ps, grep, awk, sed, ss, ip)

E1.1 – Processi e thread

Scrivere una pipeline che mostri i primi 3 processi ordinati per numero di thread (LWP) decrescente, mostrando:

PID NUM_LWP COMMAND

Suggerimenti:

- a. comandi utili: ps, sort, head
- b. ps -eo pid,nlwp,comm

PID	NLWP	COMMAND
1	1	systemd
2	1	kthreadd
3	1	rcu_gp
345	4	gnome-shell

Esercitazione 2

SEZIONE 1 – Pipeline di comandi (ps, grep, awk, sed, ss, ip)

E1.1 – Processi e thread

Scrivere una pipeline che mostri i primi 3 processi ordinati per numero di thread (LWP) decrescente, mostrando:

PID NUM_LWP COMMAND

Soluzione:

```
ps -eo pid,nlwp,comm | sort -k2 -nr | head -3
```


Esercitazione 2

SEZIONE 1 – Pipeline di comandi (ps, grep, awk, sed, ss, ip)

E1.1b – Processi e thread

Scrivere uno script che mostri i primi N processi ordinati per numero di thread (LWP) decrescente, mostrando:

PID NUM_LWP COMMAND

Esercitazione 2

SEZIONE 1 – Pipeline di comandi (ps, grep, awk, sed, ss, ip)

E1.1c – Processi e thread

Scrivere uno script che mostri il pid dei processi con il massimo numero di thread

Suggerimenti:

1. Usare ps, sort, awk
2. Partire da: ps -eo pid,nlwp
3. Usare awk per selezionare i massimi

Esercitazione 2

SEZIONE 1 – Pipeline di comandi (ps, grep, awk, sed, ss, ip)

E1.1c – Processi e thread

Scrivere uno script che mostri il pid dei processi con il massimo numero di thread

Suggerimenti:

1. Usare ps, sort, awk
2. Partire da: ps -eo pid,nlwp
3. Usare awk per selezionare i massimi

Soluzione:

```
ps -eo pid,nlwp | sort -k2 -nr | awk 'NR==1 {max=$2} $2==max {print $1}'
```

Esercitazione 2

SEZIONE 1 – Pipeline di comandi (ps, grep, awk, sed, ss, ip)

E1.1c – File ed espressioni

Scrivere una pipeline di comandi che individua il file regolare più grande nella directory corrente (cioè quello con dimensione in byte massima), visualizza il contenuto del file sostituendo tutto il contenuto delle parentesi tonde con x

Suggerimenti:

1. Usare ls, cat, head, awk, e sed

2. Partire da ls -l

```
drwxr-xr-x 2 alberto alberto 4096 Sep 13 12:31 Desktop
drwxr-xr-x 4 alberto alberto 4096 Sep 16 13:32 Documents
drwxr-xr-x 2 alberto alberto 4096 Sep 13 12:31 Downloads
```

Esercitazione 2

SEZIONE 1 – Pipeline di comandi (ps, grep, awk, sed, ss, ip)

E1.1c – File ed espressioni

Scrivere una pipeline di comandi che individua il file regolare più grande nella directory corrente (cioè quello con dimensione in byte massima), visualizza il contenuto del file sostituendo tutto il contenuto delle parentesi tonde con x

Suggerimenti:

1. Usare ls, cat, head, awk, e sed

2. Partire da ls -l

```
drwxr-xr-x 2 alberto alberto 4096 Sep 13 12:31 Desktop
drwxr-xr-x 4 alberto alberto 4096 Sep 16 13:32 Documents
drwxr-xr-x 2 alberto alberto 4096 Sep 13 12:31 Downloads
```

Soluzione:

```
cat "$(ls -l | grep '^-' | sort -k5 -nr | awk '{print $9}' | head -1)" | sed 's/\([^)]*\)/xxx/g'
```

Esercitazione 2

SEZIONE 2 – Thread & Sync

E2.1 – Cond. Var.

Si consideri il seguente programma C che crea **due thread**, A e B, che devono stampare in ordine:

A

B

A

B

A

B

...

per un totale di 10 stampe (5 volte A, 5 volte B), alternandosi.

Completare il codice riempiendo le parti // TODO usando mutex e condition variable per garantire la turnazione corretta tra i due thread.

Esercitazione 2

SEZIONE 2 – Thread & Sync

E2.2 – Analisi Cond. Var.

```
#include <stdio.h>

#include <pthread.h>
#include <unistd.h>

pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;
int ready = 0;

void* worker(void* arg) {
    pthread_mutex_lock(&m);
    while (!ready)
        pthread_cond_wait(&c, &m);
    printf("OK\n");
    pthread_mutex_unlock(&m);
    return NULL;
}
```

Esercitazione 2

SEZIONE 2 – Thread & Sync

E2.2 – Analisi Cond. Var.

```
int main(void) {  
    pthread_t t;  
    pthread_create(&t, NULL, worker, NULL);  
    pthread_mutex_lock(&m);  
    // simuliamo un po' di lavoro  
    sleep(2);  
    pthread_mutex_lock(&m);  
    ready = 1;  
    pthread_cond_signal(&c);  
  
    pthread_join(t, NULL);  
    return 0;  
}
```

- Il programma stampa sempre OK?
- Il codice è corretto?
- Come andrebbe corretto?

Esercitazione 2

SEZIONE 2 – Pipe & Select

E2.3 – pipe

- Completare il seguente programma C che:
 - crea una pipe
 - il padre **scrive periodicamente** nella pipe
 - il figlio usa select() per attendere **sia**:
 - dati da stdin
 - dati dalla pipe
 - e stampa con prefisso:
 - "PIPE: ..." per ciò che arriva dalla pipe
 - "STDIN: ..." per ciò che digita l'utente
- Completare le parti marcate // TODO.

Esercitazione 2

SEZIONE 2 – Socket & Select

E2.3 – chat

- Completare un programma C che realizza una piccola **chat a due**:
 - il programma si connette a un altro peer (porta locale, IP-peer e porta-peer da riga di comando)
 - in un ciclo infinito usa select() per attendere:
 - dati da stdin (utente),
 - dati dalla socket con il peer;
 - se l'utente scrive una riga, il programma la invia al peer;
 - se il peer invia dati, il programma li stampa come:
 - lui dice: <messaggio>
- Completare le parti marcate // TODO.